МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ «МИСИС»

ИНСТИТУТ КОМПЬЮТЕРНЫХ НАУК (ИКН)

КАФЕДРА АВТОМАТИЗИРОВАННЫХ СИСТЕМ УПРАВЛЕНИЯ (АСУ)

Курс «Сетевые технологии»

Отчёт по лабораторной работе на тему:

«ЛР-8. Отработка комплексных практических навыков»

Выполнил: студент группы БИВТ-24-7

Бут Иван Антонович

Проверил: КТН, СНС, доцент

Микитенко Игорь Иванович

МОСКВА 2026

Цель работы

Настроить небольшую сеть в Cisco Packet Tracer по заданной топологии, корректно ввести параметры IPv4 и IPv6 на нужных устройствах, а потом проверить связность между узлами и доступ к сайтам "Accounting.pka" и "Website.pka".

Основные задачи лабораторной работы

1. Заполнить таблицу адресации по данным из задания.
2. Настроить на маршрутизаторе R1 интерфейсы G0/0 и G0/1 для IPv4 и IPv6.
3. Задать на коммутаторе S1 адрес управления IPv4 и шлюз по умолчанию.
4. Настроить на узле ManagerA статические параметры IPv4 и IPv6.
5. Проверить ICMP-связность и открытие веб-страниц на обоих серверах.

Исходные данные

По условию работы оборудование уже было расставлено и физически соединено. Мне оставалось закончить адресацию, проверить подключение между двумя подсетями и убедиться, что оба сайта открываются из браузера ManagerA. В отчете ниже я иду по шагам: сначала адресация, потом настройка устройств, затем проверка сети и дополнительное исследование.

Таблица адресации

Устройство	Интерфейс	Адрес	Маска / префикс	Шлюз по умолчанию
R1	G0/0	172.16.10.1	255.255.255.192	-
		2001:DB8:CAFE:1::1	/64	-
R1	G0/1	172.16.10.65	255.255.255.192	-
		2001:DB8:CAFE:2::1	/64	-
R1	Link-local	FE80::1	-	-
S1	VLAN1	172.16.10.62	255.255.255.192	172.16.10.1
S2	VLAN1	172.16.10.126	255.255.255.192	172.16.10.65
ManagerA	NIC	172.16.10.3	255.255.255.192	172.16.10.1
		2001:DB8:CAFE:1::3	/64	FE80::1
Accounting.pka	NIC	172.16.10.2	255.255.255.192	172.16.10.1
		2001:DB8:CAFE:1::2	/64	FE80::1
ManagerB	NIC	172.16.10.67	255.255.255.192	172.16.10.65
		2001:DB8:CAFE:2::3	/64	FE80::1
Website.pka	NIC	172.16.10.66	255.255.255.192	172.16.10.65
		2001:DB8:CAFE:2::2	/64	FE80::1

Примечание. Для хостов в отчете приведены фактически используемые шлюзы по умолчанию: для IPv4 это адрес интерфейса R1 в своей подсети, для IPv6 - link-local адрес FE80::1.

Ход выполнения работы

1. Заполнение таблицы адресации

Сначала я просто перенес значения из задания в общую таблицу. На этом шаге важно было не перепутать две подсети: левую сеть с адресами вида 172.16.10.x и правую сеть с адресами, начинающимися с 172.16.10.65. Отдельно я отметил link-local адрес маршрутизатора, потому что он потом пригодился для IPv6-шлюза.

После заполнения таблицы работать дальше стало заметно проще: перед глазами уже были все IPv4-адреса, маски, IPv6-префиксы и адреса шлюзов.

2. Настройка маршрутизатора R1

Дальше я перешел к маршрутизатору R1. На интерфейсе G0/0 задал адреса для левого сегмента, на G0/1 - для правого. После этого включил IPv6-маршрутизацию и сохранил конфигурацию.

Логи

```
R1>enable
R1#conf t
R1(config)#interface g0/0
R1(config-if)#ip address 172.16.10.1 255.255.255.192
R1(config-if)#ipv6 address 2001:DB8:CAFE:1::1/64
R1(config-if)#ipv6 address FE80::1 link-local
R1(config-if)#no shutdown
R1(config)#interface g0/1
R1(config-if)#ip address 172.16.10.65 255.255.255.192
R1(config-if)#ipv6 address 2001:DB8:CAFE:2::1/64
R1(config-if)#ipv6 address FE80::1 link-local
R1(config-if)#no shutdown
R1(config)#ipv6 unicast-routing
R1(config)#end
R1#copy running-config startup-config
```

После ввода основных команд я отдельно проверил состояние интерфейсов. Для меня это был самый удобный быстрый тест: если на этом этапе порт не в состоянии up/up, дальше можно даже не идти.

```
R1#show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0     172.16.10.1     YES manual up                    up
GigabitEthernet0/1     172.16.10.65    YES manual up                    up
GigabitEthernet0/2     unassigned      YES unset  administratively down down
Serial0/0/0            unassigned      YES unset  down                  down
Serial0/0/1            unassigned      YES unset  administratively down down
Vlan1                  unassigned      YES unset  administratively down down
```

По выводу видно, что оба рабочих интерфейса маршрутизатора поднялись нормально. Это значит, что R1 уже готов пересылать трафик между двумя сегментами, а IPv6 тоже будет работать, потому что команда ipv6 unicast-routing включена.

3. Настройка коммутатора S1

На S1 нужно было задать адрес управления на интерфейсе VLAN1. S2 по условию уже был настроен, поэтому его я не трогал. Тут все довольно коротко: адрес, маска, no shutdown и шлюз по умолчанию.

Логи

```
S1>enable
S1#conf t
S1(config)#interface vlan 1
S1(config-if)#ip address 172.16.10.62 255.255.255.192
```

```
S1(config-if)#no shutdown
S1(config)#ip default-gateway 172.16.10.1
S1(config)#end
S1#copy running-config startup-config
```

Отдельно важно, что шлюз по умолчанию указывает именно на 172.16.10.1, то есть на левый интерфейс R1. После этого коммутатор стал доступен как управляемое устройство из своей подсети.

4. Настройка узла ManagerA

Узел ManagerA я настраивал через окно Desktop -> IP Configuration. Для IPv4 выбрал Static и ввел адрес, маску и шлюз. Для IPv6 тоже поставил Static, указал глобальный адрес и link-local шлюз FE80::1. Во время проверки Activity пришлось внимательно заполнить поля шлюза и DNS, потому что Packet Tracer засчитывал не просто "работает", а именно ожидаемые значения.

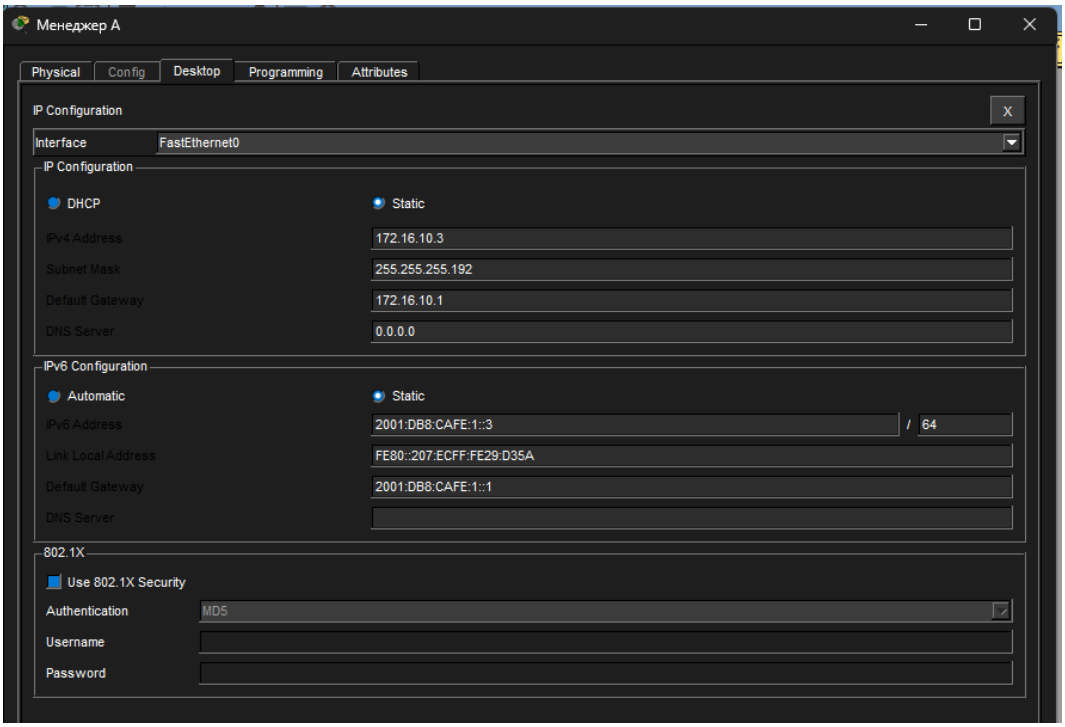

Рисунок 1 - окно IP Configuration на ManagerA

На этом шаге получилась полноценная конфигурация хоста: IPv4-адрес 172.16.10.3, IPv4-шлюз 172.16.10.1, IPv6-адрес 2001:DB8:CAFE:1::3/64 и IPv6-шлюз FE80::1. После такой настройки ManagerA уже может обращаться и к своей сети, и к удаленному сегменту через R1.

5. Проверка связности по IPv4

Сначала я проверил ближайшие устройства: сам шлюз и адрес управления S1. Это хороший порядок, потому что сразу видно, работает ли локальный сегмент до выхода в соседнюю подсеть.

Логи

```
C:\>ping 172.16.10.1
Pinging 172.16.10.1 with 32 bytes of data:
Reply from 172.16.10.1: bytes=32 time<1ms TTL=255
Reply from 172.16.10.1: bytes=32 time<1ms TTL=255
Reply from 172.16.10.1: bytes=32 time<1ms TTL=255
Reply from 172.16.10.1: bytes=32 time<1ms TTL=255
Ping statistics for 172.16.10.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Minimum = 0ms, Maximum = 0ms, Average = 0ms

C:\>ping 172.16.10.62
Pinging 172.16.10.62 with 32 bytes of data:
Request timed out.
Reply from 172.16.10.62: bytes=32 time<1ms TTL=255
Reply from 172.16.10.62: bytes=32 time<1ms TTL=255
Reply from 172.16.10.62: bytes=32 time<1ms TTL=255
```

Первый таймаут при обращении к S1 здесь не выглядит ошибкой. Это обычная ситуация, когда устройство сначала выполняет ARP-процедуру, а уже потом начинает стабильно отвечать на эхо-запросы.

```
C:\>ping 172.16.10.2
Pinging 172.16.10.2 with 32 bytes of data:
Reply from 172.16.10.2: bytes=32 time<1ms TTL=128
Reply from 172.16.10.2: bytes=32 time<1ms TTL=128
Reply from 172.16.10.2: bytes=32 time<1ms TTL=128
Reply from 172.16.10.2: bytes=32 time<1ms TTL=128

C:\>ping 172.16.10.66
Pinging 172.16.10.66 with 32 bytes of data:
Request timed out.
Reply from 172.16.10.66: bytes=32 time<1ms TTL=127
Reply from 172.16.10.66: bytes=32 time<1ms TTL=127
Reply from 172.16.10.66: bytes=32 time<1ms TTL=127

C:\>ping 172.16.10.67
Pinging 172.16.10.67 with 32 bytes of data:
Request timed out.
Reply from 172.16.10.67: bytes=32 time<1ms TTL=127
Reply from 172.16.10.67: bytes=32 time<1ms TTL=127
Reply from 172.16.10.67: bytes=32 time=1ms TTL=127
```

По IPv4 связность получилась нормальной и внутри своей подсети, и через маршрутизатор до удаленного сегмента. TTL 127 у правых узлов как раз показывает, что пакет прошел через один маршрутизатор.

6. Проверка связности по IPv6

После IPv4 я отдельно проверил работу IPv6. Здесь меня интересовали три точки: сам интерфейс R1 в левой сети, локальный сервер Accounting.pka и сервер Website.pka в соседнем сегменте.

Логи

```
C:\>ping 2001:DB8:CAFE:1::1
Pinging 2001:DB8:CAFE:1::1 with 32 bytes of data:
Reply from 2001:DB8:CAFE:1::1: bytes=32 time<1ms TTL=255
Reply from 2001:DB8:CAFE:1::1: bytes=32 time<1ms TTL=255
Reply from 2001:DB8:CAFE:1::1: bytes=32 time<1ms TTL=255
Reply from 2001:DB8:CAFE:1::1: bytes=32 time<1ms TTL=255

C:\>ping 2001:DB8:CAFE:1::2
Reply from 2001:DB8:CAFE:1::2: bytes=32 time<1ms TTL=128
Reply from 2001:DB8:CAFE:1::2: bytes=32 time<1ms TTL=128
Reply from 2001:DB8:CAFE:1::2: bytes=32 time<1ms TTL=128
Reply from 2001:DB8:CAFE:1::2: bytes=32 time<1ms TTL=128

C:\>ping 2001:DB8:CAFE:2::2
Reply from 2001:DB8:CAFE:2::2: bytes=32 time<1ms TTL=127
Reply from 2001:DB8:CAFE:2::2: bytes=32 time=1ms TTL=127
Reply from 2001:DB8:CAFE:2::2: bytes=32 time<1ms TTL=127
Reply from 2001:DB8:CAFE:2::2: bytes=32 time=1ms TTL=127
```

Если бы на R1 не была включена команда ipv6 unicast-routing, до удаленного IPv6-узла в правой подсети ответы бы не пришли. Здесь же все отработало корректно, значит и адреса, и маршрутизация настроены правильно.

7. Проверка веб-доступа и результата Activity

Последним шагом я открыл обе страницы из браузера ManagerA. Это уже удобная прикладная проверка: не просто пинги, а именно доступ к сервису на сервере.

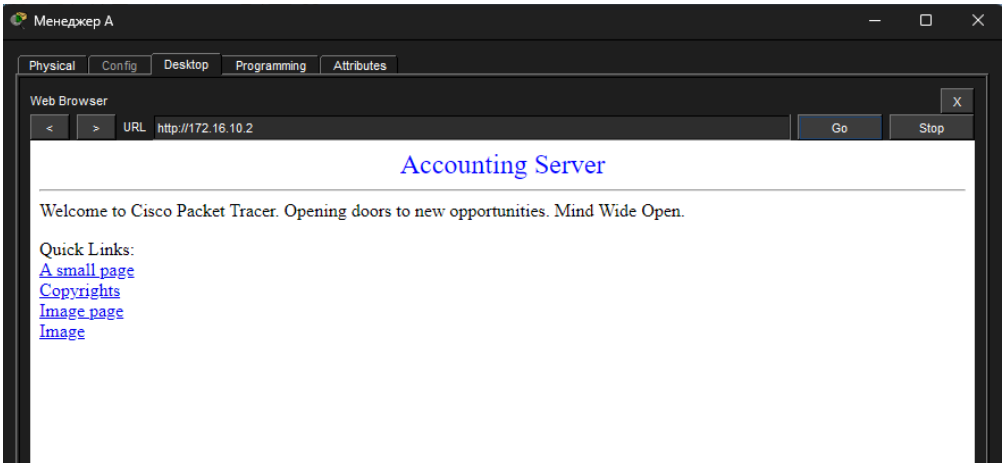

Рисунок 2 - открытие страницы сервера "Accounting.pka"

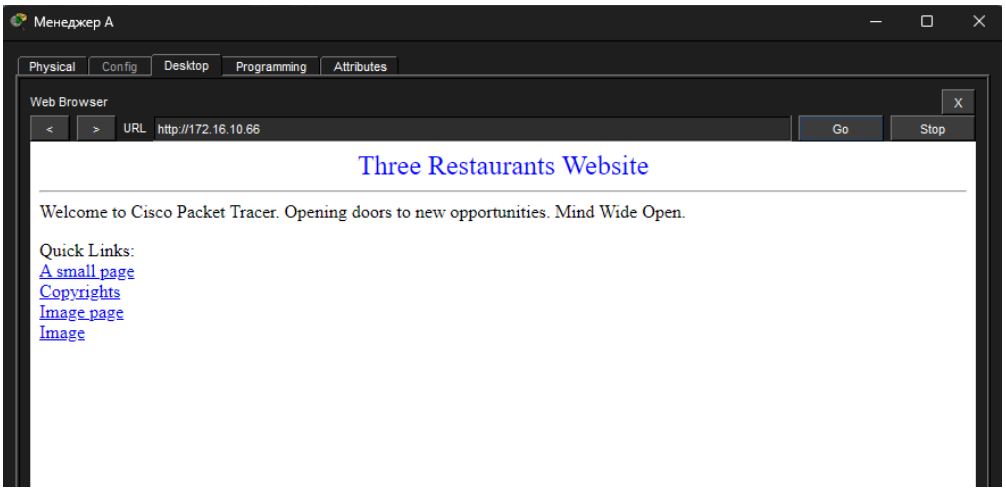

Рисунок 3 - открытие страницы сервера "Website.pka"

После исправления параметров, которые требовала внутренняя проверка Packet Tracer, активность была засчитана полностью.

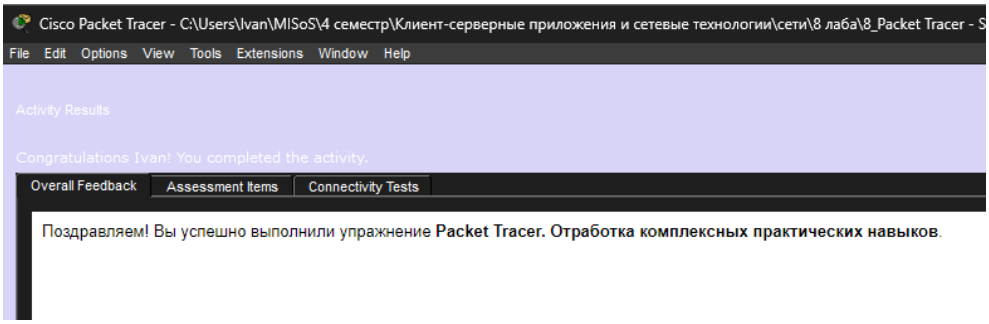

Рисунок 4 - итоговая проверка Activity, лабораторная работа выполнена полностью

В основной части работы сеть заработала так, как требовалось по заданию. Оба сегмента связаны через R1, веб-страницы доступны, а Packet Tracer подтверждает полное выполнение сценария.

Дополнительное задание. Межвлановая маршрутизация по схеме Router-on-a-Stick

Для дополнительного исследования я выбрал тему среднего уровня сложности: разделение сети на несколько VLAN и последующую маршрутизацию между ними через один физический интерфейс маршрутизатора. Этот вариант показался самым полезным, потому что здесь сразу видно и работу коммутатора, и работу роутера.

Цель исследования

Создать на коммутаторе три VLAN: SALES, ACCOUNTING и MANAGEMENT, назначить им отдельные access-порты, затем обеспечить обмен трафиком между этими VLAN через маршрутизатор по схеме Router-on-a-Stick.

1. Создание VLAN и назначение access-портов на S1

Сначала я разбил сеть на три отдельные VLAN. После этого каждый ПК оказался в своем логическом сегменте: Fa0/1 в VLAN 10, Fa0/2 в VLAN 20 и Fa0/3 в VLAN 30.

Логи

```
S1>enable
S1#conf t
S1(config)#vlan 10
S1(config-vlan)#name SALES
S1(config-vlan)#exit
S1(config)#vlan 20
S1(config-vlan)#name ACCOUNTING
S1(config-vlan)#exit
S1(config)#vlan 30
S1(config-vlan)#name MANAGEMENT
S1(config-vlan)#exit
S1(config)#interface fa0/1
S1(config-if)#switchport mode access
S1(config-if)#switchport access vlan 10
S1(config-if)#exit
S1(config)#interface fa0/2
S1(config-if)#switchport access vlan 20
S1(config-if)#exit
S1(config)#interface fa0/3
S1(config-if)#switchport mode access
S1(config-if)#switchport access vlan 30
S1(config-if)#exit
```

Чтобы убедиться, что VLAN действительно созданы и порты попали куда нужно, я посмотрел короткий сводный вывод show vlan brief.

```
S1#show vlan brief
VLAN Name                             Status    Ports
1    default                          active    Fa0/4, Fa0/5, Fa0/6, Fa0/7, Fa0/24
10   SALES                            active    Fa0/1
20   ACCOUNTING                       active    Fa0/2
30   MANAGEMENT                       active    Fa0/3
```

На этом этапе широковещательные домены уже разделены. Узлы в разных VLAN не могут обмениваться трафиком напрямую на уровне L2, и это как раз то, что нам было нужно перед настройкой маршрутизации.

2. Настройка trunk между S1 и R1

Дальше я перевел порт Fa0/24, который соединяет S1 с маршрутизатором, в режим trunk. Через этот порт должны идти кадры сразу нескольких VLAN с метками 802.1Q.

Логи

```
S1#conf t
S1(config)#interface fa0/24
S1(config-if)#switchport mode trunk
S1(config-if)#switchport trunk allowed vlan 10,20,30
S1(config-if)#exit
S1(config)#end
S1#copy running-config startup-config
```

Сразу после настройки я посмотрел подробный режим порта. На промежуточном шаге было видно, что Administrative Mode уже trunk, а Operational Mode еще down. Это нормальный момент, пока с другой стороны не завершена настройка физического интерфейса на R1.

```
S1#show interfaces fa0/24 switchport
Name: Fa0/24
Administrative Mode: trunk
Operational Mode: down
Administrative Trunking Encapsulation: dot1q
Operational Trunking Encapsulation: dot1q
Trunking VLANs Enabled: 10,20,30
```

Здесь в моем случае проблема была не в самом trunk, а в том, что физический интерфейс маршрутизатора еще не был окончательно поднят под схему Router-on-a-Stick.

3. Настройка R1 для Router-on-a-Stick

На маршрутизаторе я убрал IPv4-адрес с физического интерфейса G0/0 и создал три подинтерфейса: G0/0.10, G0/0.20 и G0/0.30. Каждый из них получил свой адрес и свою метку VLAN.

Логи

```
R1>enable
R1#conf t
R1(config)#interface g0/0
R1(config-if)#no ip address
R1(config-if)#no shutdown
R1(config-if)#exit
R1(config)#interface g0/0.10
R1(config-subif)#encapsulation dot1Q 10
R1(config-subif)#ip address 192.168.10.1 255.255.255.0
R1(config-subif)#exit
R1(config)#interface g0/0.20
R1(config-subif)#encapsulation dot1Q 20
R1(config-subif)#ip address 192.168.20.1 255.255.255.0
R1(config-subif)#exit
R1(config)#interface g0/0.30
R1(config-subif)#encapsulation dot1Q 30
R1(config-subif)#ip address 192.168.30.1 255.255.255.0
R1(config-subif)#end
```

После этого я проверил состояние интерфейсов. Здесь уже хорошо видно, что физический G0/0 поднят, а подинтерфейсы получили свои адреса и тоже находятся в состоянии up/up.

```
R1#show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0     unassigned      YES manual up                    up
GigabitEthernet0/0.10  192.168.10.1    YES manual up                    up
GigabitEthernet0/0.20  192.168.20.1    YES manual up                    up
GigabitEthernet0/0.30  192.168.30.1    YES manual up                    up
```

Смысл команды encapsulation dot1Q здесь очень простой: она привязывает конкретную VLAN к конкретному подинтерфейсу. Без этой команды маршрутизатор не смог бы понять, к какой VLAN относится трафик, пришедший по trunk-линии.

4. Настройка хостов в трех VLAN

После маршрутизатора оставалось задать статические адреса трем ПК. Для каждого узла я указал свой IPv4-адрес, маску /24 и шлюз по умолчанию, который совпадает с адресом соответствующего подинтерфейса R1.

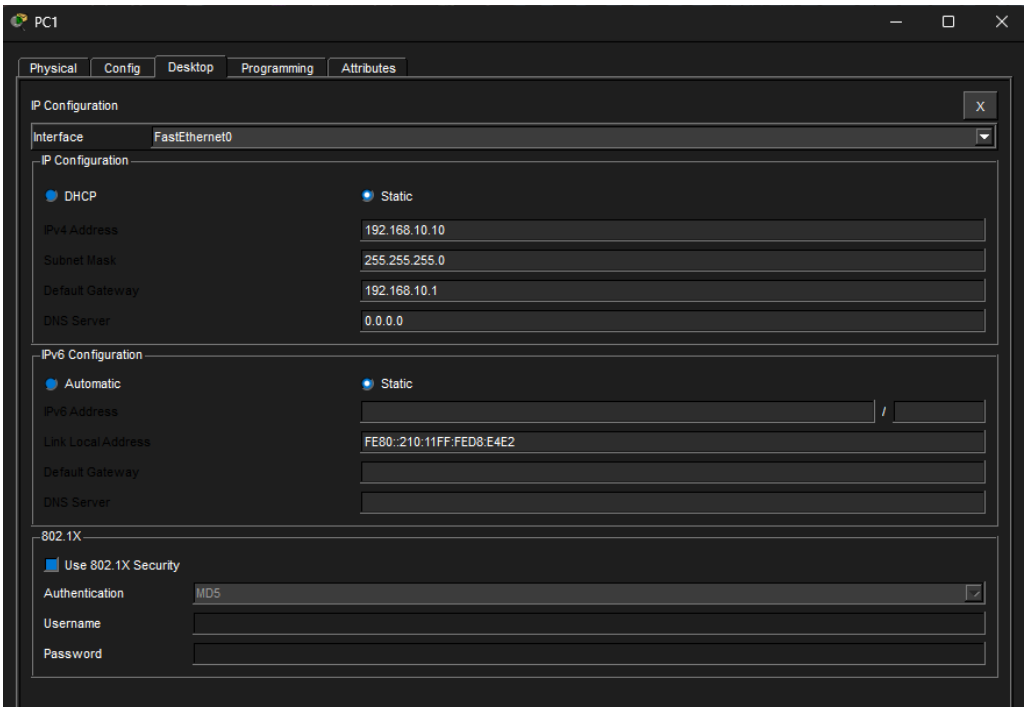

Рисунок 5 - настройки PC1, VLAN 10

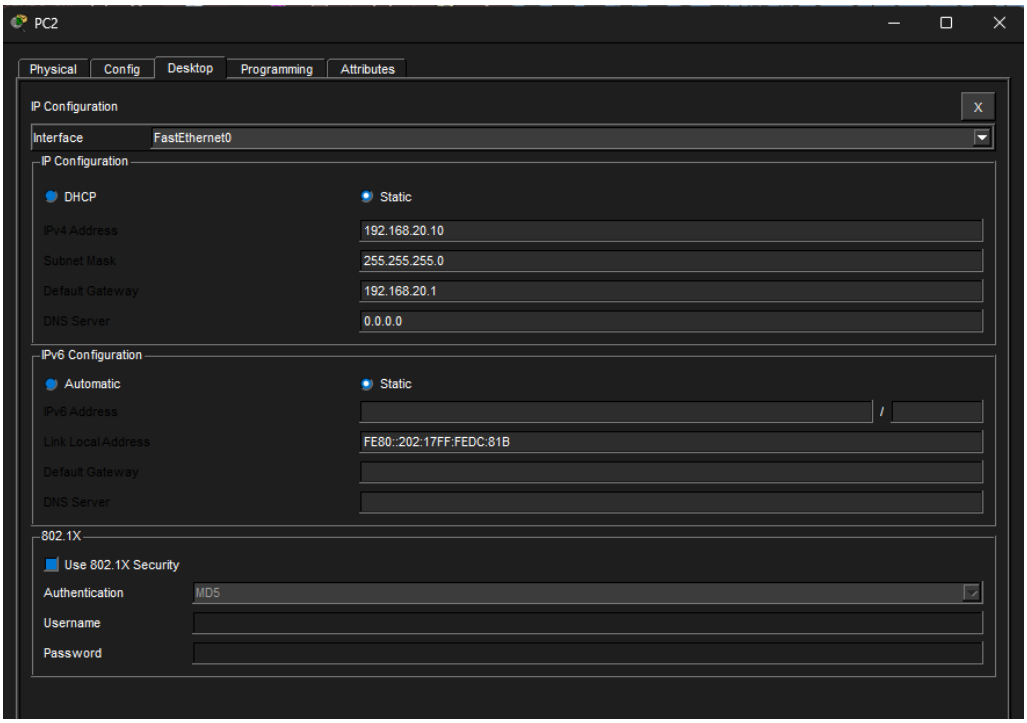

Рисунок 6 - настройки PC2, VLAN 20

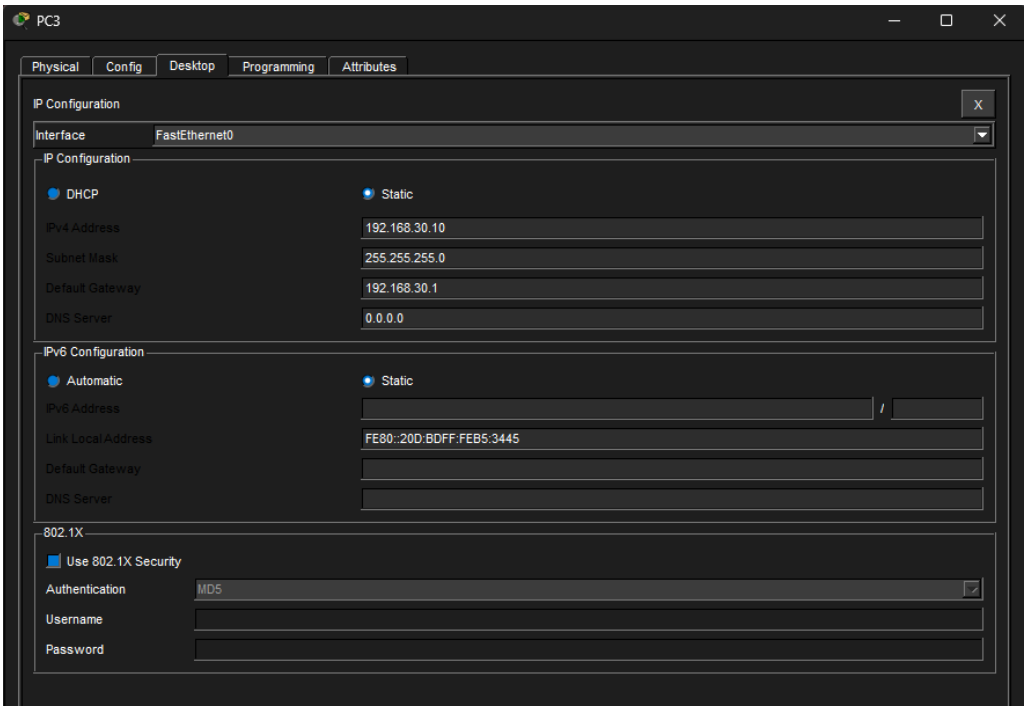

Рисунок 7 - настройки PC3, VLAN 30

После этого каждый ПК оказался в своем логическом сегменте, но уже с понятным шлюзом, через который можно идти в соседние VLAN.

5. Проверка связности между VLAN

Финальная проверка здесь самая показательная: пинг из VLAN 10 сначала в VLAN 20, а потом в VLAN 30. Первый пакет местами терялся, но это ожидаемо из-за ARP. Повторный запуск уже дал стабильный ответ.

Логи

```
C:\>ping 192.168.20.10
Pinging 192.168.20.10 with 32 bytes of data:
Request timed out.
Reply from 192.168.20.10: bytes=32 time=3ms TTL=127
Reply from 192.168.20.10: bytes=32 time=1ms TTL=127
Reply from 192.168.20.10: bytes=32 time<1ms TTL=127

C:\>ping 192.168.20.10
Reply from 192.168.20.10: bytes=32 time<1ms TTL=127
Reply from 192.168.20.10: bytes=32 time<1ms TTL=127
Reply from 192.168.20.10: bytes=32 time=1ms TTL=127
Reply from 192.168.20.10: bytes=32 time=1ms TTL=127
C:\>ping 192.168.30.10
Pinging 192.168.30.10 with 32 bytes of data:
Request timed out.
Reply from 192.168.30.10: bytes=32 time=1ms TTL=127
Reply from 192.168.30.10: bytes=32 time=2ms TTL=127
Reply from 192.168.30.10: bytes=32 time<1ms TTL=127

C:\>ping 192.168.30.10
Reply from 192.168.30.10: bytes=32 time<1ms TTL=127
Reply from 192.168.30.10: bytes=32 time=3ms TTL=127
```

```
Reply from 192.168.30.10: bytes=32 time=1ms TTL=127
Reply from 192.168.30.10: bytes=32 time<1ms TTL=127
```

По этим пингам видно главное: межвлановая маршрутизация работает. Узлы из разных VLAN не сидят в одном L2-сегменте, но через R1 спокойно обмениваются трафиком. Для небольшой сети это очень наглядная и удобная схема.

Итог дополнительного исследования

В дополнительной части я не просто создал VLAN, а довел конфигурацию до полноценной межвлановой маршрутизации. На практике получилось увидеть весь путь целиком: access-порты на коммутаторе, trunk на линии к R1, подинтерфейсы с encapsulation dot1Q и успешные пинги между узлами из разных VLAN. Для меня это было полезно тем, что стало понятно, где заканчивается изоляция уровня L2 и где уже начинается маршрутизация на уровне L3.

Общий вывод по лабораторной работе

По основной части лабораторной работы я закрепил настройку статической адресации в Packet Tracer. На R1 удалось корректно ввести IPv4 и IPv6 для двух интерфейсов, на S1 - адрес управления, а на ManagerA - параметры хоста. Проверка пингами показала, что связь есть и внутри своего сегмента, и через маршрутизатор до соседней подсети. Отдельно через браузер подтвердился доступ к двум веб-серверам, а внутренняя проверка Activity засчитала работу полностью.

Дополнительное исследование дало уже более практическую картину по VLAN. Я увидел, как одна и та же физическая линия между коммутатором и маршрутизатором может переносить трафик сразу нескольких VLAN, если использовать trunk и 802.1Q. Заодно стало понятнее, почему первый ping иногда теряется: это не ошибка маршрутизации, а обычная подготовка ARP-таблицы.

Если коротко собрать результат по всей работе, то вывод такой. В основной части я отработал базовую настройку адресации и проверку сервисов, а в дополнительной - уже построил чуть более взрослую схему с VLAN и Router-on-a-Stick. Вместе эти два блока хорошо показывают, как в одной сети сочетаются настройка конечных узлов, работа маршрутизатора и логическое разделение сегментов.